

Relevance Without Repair: Retrieval Gains Are Fragile and Do Not Convert in LLM Program Repair

Do Dinh Phuc
VinUniversity
Hanoi, Vietnam
24phuc.dd@vinuni.edu.vn

Abstract—Retrieval-augmented generation (RAG) is widely assumed to improve LLM-based automated program repair: retrieve similar past fixes, show them to the model, repair more bugs. We test this assumption with a critical empirical study built around *structured failure-aware retrieval*, a reranking method that matches a structured representation of the observed test failure against repair metadata inferred from each corpus example’s fix. Across four benchmarks (QuixBugs, HumanEvalFix, a contamination-controlled MBPP repair split, and real repository bugs from PyBugHive) and six models we find: (1) structured reranking significantly improves a reranker-independent retrieval-relevance metric on QuixBugs ($p=0.0001$) but the gain does not generalize to a larger benchmark ($n=187$, $p=0.41$); (2) no retrieval variant significantly improves repair over a no-retrieval baseline in any setting — on the synthetic and algorithmic benchmarks, effects are neutral-to-slightly-negative (-0.5 to -3.7 *pp*, equivalent to baseline within ± 10 *pp*); (3) a planted-corpus experiment that guarantees a perfect example is present decomposes the failure: repair then improves by $+21.9/+11.8$ *pp* over baseline ($p<0.001$, two model families), the gain is attributable primarily to coverage rather than ranking ($+14.4/+13.4$ *pp* of it survives forcing the best *real* example), plain dense retrieval already surfaces the planted example 100% of the time, and structured reranking selects it *less* often (94.1%). On synthetic bugs, the binding constraint is *corpus coverage*, not retrieval ranking. (4) On *real repository bugs* (116 attempted cases from 10 PyBugHive projects; 55, spanning 6 projects, evaluable end-to-end in CI), an encouraging single-project pilot ($+10.7$ *pp* from planting exact fix pairs) *does not survive replication*: pooled, planting yields 70.9% vs 76.4% baseline (4 converted, 7 regressed, $p=0.55$; 95% CI $[-17.2, +6.3]$ *pp*, excluding the pilot’s effect size). Real-bug transfer of the coverage effect is thus unestablished, and our design could not have detected effects below ~ 16 *pp*. Our results caution that better retrieval relevance is neither robustly achievable nor sufficient for better repair in single-shot pipelines, and that the repository-scale mechanics downstream of retrieval deserve the attention that retrieval scoring currently receives.

Index Terms—automated program repair, retrieval-augmented generation, large language models, empirical study

I. INTRODUCTION

Large language models (LLMs) repair a substantial fraction of buggy programs from a single prompt containing the code and a failing test’s output. A natural next step, borrowed from retrieval-augmented generation, is to also show the model *similar past fixes*, and several recent systems report gains from this recipe [1]–[4]. This paper reports a critical empirical

study of that intuition. We built a strong retrieval signal for the repair setting — *structured failure-aware retrieval*, which parses the observed test failure into a structured state and reranks dense-retrieved candidates by compatibility with repair metadata mined from each corpus example — and asked four questions:

RQ1 (Relevance): does structured reranking retrieve more fix-relevant examples than dense code retrieval? **RQ2 (Conversion)**: do more relevant examples produce more successful repairs? **RQ3 (Mechanism)**: when retrieval fails to help, is the binding constraint the ranking or the corpus itself? **RQ4 (Transfer)**: do the answers transfer from synthetic benchmark bugs to real repository bugs?

Our answers, drawn from four benchmarks and six models (GPT-4o/4.1/4o-mini, Llama-3-8B, Llama-3.3-70B, Qwen2.5-7B; the benchmark $\times$ model grid is sparse, not factorial — see Section IV), are largely negative and, we argue, more useful than another reported win. The relevance gain is real but fragile: significant on QuixBugs on a reranker-independent metric ($p=0.0001$) yet null on a larger benchmark ($n=187$, $p=0.41$). Relevance does not convert: no retrieval variant *significantly* beats the no-retrieval baseline in any comparison, and equivalence testing bounds any undetected effect within ± 10 *pp* on the synthetic benchmarks. A planted-corpus experiment locates the mechanism there: with a perfect example guaranteed present, repair jumps $+21.9$ *pp*, plain dense retrieval surfaces that example 100% of the time, and our reranker selects it *less* often — on synthetic bugs the binding constraint is *corpus coverage*, and reranking sophistication is downstream-inert. On real repository bugs, that conclusion could not be confirmed: a CI replication across every supported PyBugHive case finds planting the exact fix pair null-to-slightly-negative ($p=0.55$), the 95% confidence interval excludes the effect size of our own encouraging single-project pilot, and the design’s minimum detectable effect (~ 16 *pp*) means small real effects in either direction remain possible. Real-bug transfer is unestablished — a finding we report with its limits rather than as a confirmed null.

Contributions. (1) A structured failure-aware retrieval method with an explicit reranking function over six failure-state fields; (2) a statistically grounded negative result with equivalence bounds and minimum detectable effects, not bare

non-significance; (3) a planted-corpus decomposition separating corpus coverage from retrieval ranking, replicated across two model families on synthetic bugs; (4) an end-to-end real-bug replication harness (10 projects, per-version containers) whose result — the pilot coverage effect does not survive replication — bounds the transfer claim and redirects attention to the repair pipeline downstream of retrieval.

II. RELATED WORK

LLM repair pipelines prompt a model with buggy code and an observed failure, validating candidates against tests; benchmarks span algorithmic bugs (QuixBugs [5]), function-level edits (HumanEvalFix from OctoPack [6]), synthetic mutations (MBPP [7]), and real repository bugs (BugsInPy [8], PyBugHive [9]). Several systems retrieve historical fixes to condition repair: InferFix [2] retrieves by static-analyzer bug type, ReCode [1] by statically inferred algorithm type, RAP-Gen [3] with a learned hybrid retriever, and RAGFix [4] retrieves Stack Overflow posts through a question-answering knowledge graph. Our signal differs: a structured state extracted from the *dynamic* test failure, with an explicit, ablatable reranking function. Direct numeric comparison is infeasible (closed source, different languages); our contribution is a controlled analysis of whether the retrieval component pays off at all.

A complementary line repairs bugs *iteratively*: conversational repair with test feedback [10], self-repair loops [11], and agentic systems that localize and edit repository code (AutoCodeRover [12], Agentless [13]) evaluated on SWE-bench [14]; CodeRAG-Bench [15] studies retrieval for code tasks broadly and likewise finds gains are uneven. We deliberately study the *single-shot* regime (one attempt, temperature 0, one candidate) to isolate the retrieval signal from iteration effects; our conclusions are bounded to that regime, and whether retrieval pays off inside agentic loops is an open question our design does not answer.

Outside repair, retrieval studies report conditions where RAG does not help: irrelevant context degrades reasoning [16], mid-context information is underused [17], and retrieval helps mainly where parametric knowledge is weak [18] — a pattern our headroom results echo. Our planted-corpus design adds a repair-specific mechanism: separating *whether the corpus contains a sufficiently relevant example* (coverage) from *whether the retriever surfaces it* (ranking), which prior repair evaluations conflate.

III. STRUCTURED FAILURE-AWARE RETRIEVAL

Pipeline. Given buggy Python code and a raw failure signal (pytest output), we (1) extract a *structured failure state*; (2) dense-retrieve candidate bug-fix pairs from a FAISS index over buggy code (all-MiniLM-L6-v2 [19]); (3) rerank; (4) prompt the LLM with the bug, the failure signal, and the top- k ($k=2$) examples rendered as compact buggy→fixed diffs plus an inferred one-line “repair lesson”; (5) apply the edit and run the tests. Small files are replaced whole; large files

(>40k chars) are edited through a strict JSON line-range patch schema with validation retry.

Failure state and repair metadata. The query side has six fields extracted from the failure signal: `failure_mode`, `exception_type`, `test_name`, `assertion_summary`, `suspicious_symbols`, `contract_tags`. Each corpus example’s diff is mined for repair metadata: `repair_pattern_tags`, `suspicious_symbols`, `changed_operators`, `edit_scope`, `file_family`, `changed_lines`.

Variants. We compare BASELINE (no retrieval), CODE_ONLY (dense retrieval, rank = dense rank), RAW_TEXT (dense over code + raw failure text), RAW_TEXT_RERANK (+ lexical failure-text reranking), and STRUCTURED, which scores each candidate at dense rank r as $1.0 \text{dense}(r) + 0.8 \text{tagOverlap} + 0.7 \text{symOverlap} + 0.8 \text{modeCompat} + 0.5 \text{excCompat} + 0.2 \text{fileFam} - \text{changedLines}/400$, with Jaccard overlaps and hand-written compatibility tables. Weights were fixed once on early QuixBugs traces and never tuned on downstream results. (An exploratory per-field ablation on QuixBugs/GPT-4o moved outcomes by one problem per removed field — within the run-to-run variance we report in Section IV, so we draw no per-field conclusions from it.)

IV. STUDY DESIGN

Benchmarks. QuixBugs [5] (40 algorithmic bugs); HumanEvalFix [6] (164 function-level tasks); **MBPP-holdout**, a contamination-controlled repair split we construct from MBPP [7] (187 synthetic single-operator mutations; 508-pair retrieval corpus with zero problem overlap); and **Py-BugHive** [9]: all 116 supported real repository bugs across 10 projects (black, pandas, jax, freqtrade, salt, spaCy, poetry, discord.py, cookiecutter, scrapy), evaluated end-to-end — check-out, per-case environment build in per-version x86 containers, patch application, full test run — via a sharded CI harness (46 jobs). 55 cases, spanning 6 projects, complete all three arms cleanly; attrition is project-correlated (usable/supported: black 26/34, pandas 23/34, salt 3/7, discord.py 1/2, cookiecutter 1/1, scrapy 1/1, and jax 0/13, freqtrade 0/12, poetry 0/8, spaCy 0/4 — lost to dependency-build failures, toolchain incompatibilities, and test-suite timeouts), so the usable set is dominated by two projects and conclusions from it inherit that skew (Section VI).

Corpora. Primary: 551 real bug-fix pairs from BugsInPy [8]. For PyBugHive we use a project-held-out variant (277 pairs, all PyBugHive projects removed) — without this control, same-project entries inflate apparent RAG effects. For MBPP we additionally build a genuine MBPP corpus (1,041 pairs) to test domain match.

Models. GPT-4o, GPT-4.1, GPT-4o-mini (OpenAI); Llama-3-8B, Llama-3.3-70B, Qwen2.5-7B (Together AI). The benchmark×model grid is sparse by design, not factorial: QuixBugs and HumanEvalFix use the GPT family (plus Llama on HumanEvalFix), MBPP-holdout uses the three non-saturated/gap models, and PyBugHive uses GPT-4o-mini only.

All runs use temperature 0, a single attempt, and a single candidate. *Within a fixed environment*, temperature-0 reruns flip $\leq 1/187$ outcomes; across environments (local vs. CI) larger variation appears (Section V-D), so we compare arms only within the same environment and run. QuixBugs results are 5-trial means where variance matters.

Measures and statistics. Retrieval relevance uses a *reranker-independent* metric: cosine similarity between a candidate’s fix diff and the ground-truth fix diff, computed with the same sentence encoder used for dense retrieval [19] but over fix diffs, which no variant embeds or optimizes — independent of the reranker objective, though not of the encoder family. (A tag-compatibility proxy overstates: +81% vs +13% on QuixBugs; we report it only as illustrative.) We additionally run a *blind human audit*: the benchmark’s first 20 problems, each variant’s top-1 retrieved example presented as anonymized candidates A–D (shuffled per problem), independently judged fix-relevant or not by two annotators. Repair success is test-suite pass. Significance: exact McNemar on paired outcomes from a single designated run per comparison; Wilcoxon + bootstrap CIs for relevance; paired-difference TOST at $\pm 5/\pm 10$ *pp* margins with minimum detectable effects (MDE) at 80% power. We run many tests and apply no global correction; instead we designate the confirmatory claims (the QuixBugs relevance gain, $p=0.0001$, and the planted-corpus gains, $p<0.001$ — both of which survive any standard correction) and treat all marginal values ($0.02<p<0.11$) as exploratory, narrating them symmetrically.

V. RESULTS

A. RQ1: Relevance — improved, but fragile

On QuixBugs ($n=40$), STRUCTURED improves the independent metric over CODE_ONLY at every depth (top-5: 0.234 vs 0.207; +0.027, 95% CI [+0.016, +0.037]; Wilcoxon $p=0.0001$; higher on 31/40). The gain *does not replicate* on MBPP-holdout ($n=187$): 0.456 vs 0.464 (-0.008 , $p=0.41$). Two interpretations fit one positive and one null: (a) benchmark-specificity — MBPP’s single-operator mutations leave a thin failure signal and dense retrieval already matches near-identical buggy code; or (b) the gain is simply not robust and the larger-sample null is the better estimate. We cannot distinguish these; either way the one clean positive is fragile.

The blind human audit is directionally consistent with the QuixBugs gain, and no more than that. Both annotators rated STRUCTURED’s top-1 example fix-relevant most often (60% and 30%, vs 45% and 25% for CODE_ONLY; 41.7% vs 28.6% on agreed judgments), and neither annotator judged any CODE_ONLY example relevant where STRUCTURED’s was not ($b=0$). But the evidence is thin: $n=20$ per variant, no pairwise difference significant except against RAW_TEXT ($c=9$, $b=1$, $p=0.021$, annotator 1 — exploratory, uncorrected), fair agreement only ($\kappa=0.36$, raw 70%; the annotators’ base rates differ two-fold), one annotator is an author, and the audit covers only the benchmark where the automated metric was

TABLE I
DOWNSTREAM REPAIR VS. NO-RETRIEVAL BASELINE (SINGLE ATTEMPT, $k=2$). ALL DIFFERENCES NS BY EXACT MCNEMAR.

Benchmark	Model	base	STRUCTURED	<i>p</i>
QuixBugs	GPT-4o	35/40	37/40	ns
QuixBugs	GPT-4.1	36/40	35/40	ns
QuixBugs	GPT-4o-mini	33/40	30/40	ns
HumanEvalFix	GPT-4o	80.5%	76.8%	0.109
HumanEvalFix	GPT-4.1	79.9%	78.7%	0.727
HumanEvalFix	GPT-4o-mini	68.3%	66.5%	0.629
HumanEvalFix	Llama-3.3-70B	69.5%	67.7%	0.648
MBPP-holdout	GPT-4o-mini	93.0%	91.4%	0.55
MBPP-holdout	Llama-3-8B	64.7%	64.2%	1.0

TABLE II
COVERAGE-VS-RANKING DECOMPOSITION ($n=187$; EXACT MCNEMAR VS BASELINE).

Model (base)	ORACLE_CORPUS	planted	rank. hr	cov. hr
Llama-3-8B (64.7%)	72.2%	86.6%	+7.5	+14.4
Qwen2.5-7B (78.6%)	77.0%	90.4%	−1.6	+13.4
GPT-4o-mini (93.0%)	94.7%	94.1%	+1.6	−0.5

positive. We treat it as weak corroboration of direction, and rest no claim on it.

B. RQ2: Conversion — neutral to slightly negative

Table I (each row: one designated temperature-0 run; McNemar on that run’s paired outcomes): no comparison is significant, STRUCTURED never *significantly* beats baseline anywhere, and every HumanEvalFix/MBPP point estimate is negative (-0.5 to -3.7 *pp*). The QuixBugs point estimates are positive (+2/GPT-4o) but within 5-trial variance (means 89.5% vs 91.0%, overlapping); CODE_ONLY and raw-text variants behave like STRUCTURED throughout. Because non-significance is not “no effect,” we add equivalence: all comparisons are equivalent to baseline within ± 10 *pp* but not ± 5 *pp*; MDEs are 4.8–9.1 *pp*. For GPT-4o on HumanEvalFix the Wald 90% CI $[-6.8, -0.5]$ excludes zero while exact McNemar ($b=8$, $c=2$) does not ($p=0.109$); with discordant counts this small we defer to the exact test and read *borderline small harm* — and we note the symmetric case: the $p=0.065$ ranking-headroom value in RQ3, which favors retrieval, deserves and receives the same exploratory, not-established reading. Controls rule out easy explanations: corpus–benchmark domain match moves results by small, inconsistent, non-significant amounts; and the Llama-3-8B prompt-format collapse (123/164 syntax-invalid outputs under the longer RAG prompt) disappears with robust extraction (0/187), after which retrieval remains neutral.

C. RQ3: Mechanism — coverage dominates ranking

We answer with two forcing experiments on MBPP-holdout. ORACLE_CORPUS bypasses retrieval and shows the model the *best real* corpus example (selected by ground-truth fix similarity): the ceiling of perfect *ranking*. The *planted corpus* instead guarantees *coverage*: each test problem’s exact buggy→fixed pair is inserted and normal retrieval runs.

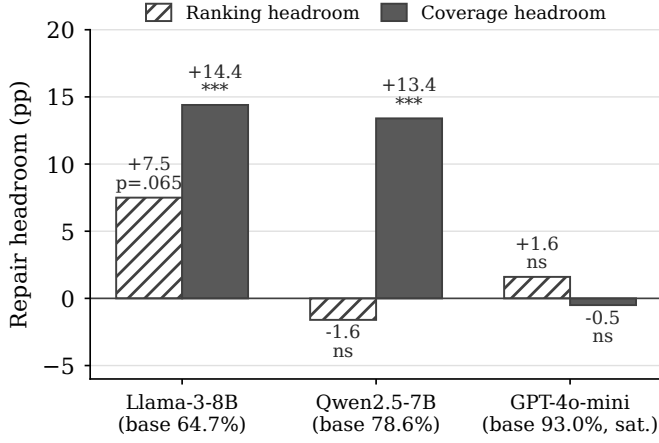


Fig. 1. Coverage headroom (planted – ORACLE_CORPUS) dwarfs ranking headroom (ORACLE_CORPUS – base) on both non-saturated models and vanishes on the saturated control. ***: $p < 0.001$, exact McNemar.

Table II and Fig. 1 give the central result. With a perfect example present, repair jumps $+21.9 pp$ (Llama-3-8B, $p < 0.0001$). Decomposed, **coverage headroom is large and significant on both non-saturated models** ($+14.4$, $+13.4 pp$; $p < 0.001$) and zero on the saturated control, while **ranking headroom is small and heterogeneous** ($+7.5 pp$, $p = 0.065$ on Llama; $-1.6 pp$, ns, on Qwen — forcing the best real example does not help). We refuse to average these: the binding constraint is *primarily corpus coverage*. Most damaging for reranking: with the perfect example present, plain dense retrieval surfaces it in the top- k 100% of the time; STRUCTURED demotes it in 11/187 cases (94.1%), with identical conversion (86.6% both). At the instance level, retrieval flips 37 of the 8B outcomes (18 wins, 19 losses) and the mean retrieved-example relevance of wins (2.645) is indistinguishable from losses (2.621). (ORACLE_SELF — showing the model its own fix pattern — reaches 90.4%, $+25.7 pp$: a sanity check that the model can exploit a near-identical example, not a deployable ceiling.)

D. RQ4: Transfer to real repository bugs — unconfirmed

Synthetic mutations make perfect examples constructible by design, so RQ3 could be a synthetic-regime artifact. We test transfer end-to-end on real PyBugHive bugs. First, with the project-held-out corpus, retrieval is exactly neutral on `black` (baseline, `CODE_ONLY`, `STRUCTURED` all 23/34), consistent with RQ2. Second, an initial single-project planted pilot was encouraging: planting the exact buggy→fixed pairs lifted GPT-4o-mini from 85.7% to 96.4% on 28 `black` cases, converting all four baseline failures ($b=1$, $c=4$, $p=0.375$, underpowered).

The replication across every supported case (116 attempted, 55 usable; Table III) **does not confirm the transfer**: planting the exact fix pair yields 70.9% vs baseline 76.4% — 4 baseline failures converted, 7 passes regressed, $p=0.549$, 95% CI on the difference $[-17.2, +6.3] pp$. That interval *excludes the pilot's* $+10.7 pp$ effect, which is the replication's one firm conclusion; at the observed discordance the design's MDE is $\sim 16 pp$, so

TABLE III
REAL-BUG REPLICATION (PYBUGHIVE, GPT-4O-MINI, 55 CASES USABLE IN ALL ARMS; EXACT MCNEMAR VS BASELINE WITHIN ONE CI RUN).

	baseline	STRUCTURED (normal)	STRUCTURED (planted)
pooled (55) vs baseline exact p	42 (76.4%) — —	39 (70.9%) $b=7$, $c=4$ 0.549	39 (70.9%) $b=7$, $c=4$ 0.549
black (26)	23	20	23
pandas (23)	15	15	12
other (6)	4	4	4

small effects in either direction remain undetectable, and we do not claim a confirmed null. Three observations bound the interpretation further. (i) The pilot itself did not reproduce: on the overlapping CI case set, `black` scores 23/26 in baseline and planted alike — cross-environment variation of the same magnitude as the arm differences, which is why we compare arms only within the run. (ii) Per-project effects are inconsistent (planting matches baseline on `black` but costs three `pandas` cases), and the usable set is 89% `black`+`pandas` after project-correlated attrition (Section IV), so the pooled estimate inherits that skew. (iii) The planted-vs-baseline contrast confounds any coverage benefit with retrieval-prompt overhead; the cleaner planted-vs-STRUCTURED (normal) contrast (same prompt shape, corpora differing only in the plants) shows identical pooled totals but different per-project compositions, and per-case analysis of the 11 discordant baseline-vs-planted cases — why exact-fix examples converted 4 yet accompanied 7 regressions — is the key open diagnostic our logs do not fully resolve. What the replication establishes is therefore modest but real: the encouraging single-project effect was not recovered at scale, and any true pooled effect of guaranteed coverage on these real bugs is bounded below $+6.3 pp$ — far from the $+14$ – $22 pp$ the synthetic decomposition would predict.

VI. DISCUSSION AND THREATS

Why doesn't relevance convert? A retrieved example helps only if it is *sufficiently* relevant to the specific fix, and realistic corpora rarely contain such an example. When one exists (planted), even plain dense retrieval finds it and repair converts; when none exists, improving the *ordering* of insufficient examples cannot manufacture the missing knowledge. Headroom scales with the fraction of currently-failed problems a perfect example would fix: $+14/+13 pp$ on the two mid-base-accuracy models, ~ 0 on the saturated one.

Cost. Measured on HumanEvalFix GPT-4o traces, the structured prompt averages $\sim 4\times$ the baseline's input tokens ($\sim 1,158$ vs ~ 300) plus embedding/index/retrieval infrastructure, for equal-or-worse accuracy — Pareto-dominated. The actionable guidance follows from RQ3 and RQ4 together: reranking sophistication has negative ROI everywhere we measured; corpus coverage is where the headroom lives on synthetic bugs, so it is the better investment — but RQ4 cautions that on real repository bugs coverage alone does

not convert, and the patch-application and validation layers deserve the attention that retrieval scoring currently gets.

Evaluation hygiene. Contamination can masquerade as retrieval gains: an uncontrolled corpus (same-project entries) flipped our PyBugHive results from neutral to strongly RAG-negative before the held-out control. RAG-repair evaluations should hold out same-project corpus entries, ablate post-processing separately from retrieval, and report equivalence bounds rather than bare non-significance.

Threats. *External validity:* the coverage decomposition (RQ3) is established on synthetic mutations, where a “perfect example” is constructible by design; the real-bug replication (RQ4) could not confirm transfer, and that result is itself bounded — 55/116 cases evaluable after project-correlated attrition, a usable set dominated by two projects, one model (GPT-4o-mini), one CI run per arm, and cross-environment variation of the same magnitude as the arm differences (we did not run repeated CI trials to quantify a flake floor). *Construct validity:* both automated relevance measures are proxies sharing the retrieval encoder; the ORACLE_CORPUS arm selects the “best real example” by that same proxy, so ranking headroom is underestimated to the extent the proxy errs — an asymmetry that would shift the decomposition toward ranking, not away from coverage. The audit is weak corroboration only (Section V). Whether the planted pairs were retrieved into the $k=2$ context on the real-bug arms is not instrumented in our CI logs — on synthetic bugs the hit rate is 94–100%, but long repository files may retrieve differently, and a low hit rate would reframe RQ4’s null as a retrieval failure rather than a conversion failure. *Internal validity:* the decomposition is one bug family, two non-saturated models plus a saturated control; direction replicates across architectures, magnitudes are benchmark-specific. *Scope:* all conclusions are bounded to single-shot, temperature-0, single-candidate, $k=2$ pipelines; iterative and agentic regimes (Section II) may convert retrieval differently. *Power:* MDEs are 4.8–9.1 *pp* on the synthetic benchmarks and ~ 16 *pp* on the real-bug replication; equivalence claims are bounded at ± 10 *pp*.

VII. CONCLUSION

We set out to make retrieval-augmented repair work better by making retrieval more relevant, and instead produced a cautionary decomposition of why that framing fails. Structured failure-aware reranking can improve a relevance proxy, but the gain is fragile across benchmarks, and even where it holds it does not convert to significantly better repair in any of our comparisons. On synthetic bugs, a planted-corpus experiment locates the binding constraint in corpus coverage — when a sufficiently relevant example exists, plain dense retrieval already surfaces it and repair improves by double digits; when it does not, no reranking sophistication helps. On real repository bugs, our replication could not confirm that the coverage effect transfers: planting exact fix pairs was null-to-slightly-negative pooled ($p=0.55$), the confidence interval excludes our own pilot’s effect size, and effects below ~ 16 *pp* remain beyond the design’s reach — so we report transfer as unestablished, with

the diagnosis of the discordant cases as the key open question. For practitioners working with single-shot repair pipelines: the scorer is not the lever; the corpus is the lever on benchmark-shaped bugs; and on real bugs the repair pipeline downstream of retrieval — patch application, localization, validation — is where the evidence says attention should go. All code, corpora-construction scripts, per-case results, and the sharded CI harness for the real-bug replication are available at <https://github.com/PHUCDODINH/failure-aware-retrieval-repair>.

REFERENCES

- [1] Y. Zhao, S. Chen, J. Zhang, and Z. Li, “Recode: Improving LLM-based code repair with fine-grained retrieval-augmented generation,” in *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM)*. ACM, 2025.
- [2] M. Jin, S. Shahriar, M. Tufano, X. Shi, S. Lu, N. Sundaresan, and A. Svyatkovskiy, “Inferfix: End-to-end program repair with LLMs,” in *Proceedings of ESEC/FSE 2023*. ACM, 2023.
- [3] W. Wang, Y. Wang, S. Joty, and S. C. H. Hoi, “RAP-Gen: Retrieval-augmented patch generation with CodeT5 for automatic program repair,” in *Proceedings of ESEC/FSE 2023*. ACM, 2023.
- [4] E. Mansur, J. Chen, M. A. Raza, and M. Wardat, “RAGFix: Enhancing LLM code repair using RAG and Stack Overflow posts,” in *Proceedings of the 2024 IEEE International Conference on Big Data (BigData)*. IEEE, 2024.
- [5] D. Lin, J. Koppel, A. Chen, and A. Solar-Lezama, “Quixbugs: a multi-lingual program repair benchmark set based on the Quixey challenge,” in *Proceedings Companion of SPLASH 2017*. ACM, 2017, pp. 55–56.
- [6] N. Muennighoff, Q. Liu, A. Zebaze, Q. Zheng, B. Hui, T. Y. Zhuo, S. Singh, X. Tang, L. von Werra, and S. Longpre, “Octopack: Instruction tuning code large language models,” *arXiv preprint arXiv:2308.07124*, 2023.
- [7] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, “Program synthesis with large language models,” *arXiv preprint arXiv:2108.07732*, 2021.
- [8] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, and E. L. Ou, “Bugsinpy: a database of existing bugs in Python programs to enable controlled testing and debugging studies,” in *Proceedings of ESEC/FSE 2020*. ACM, 2020, pp. 1556–1560.
- [9] G. Antal, N. Vándor, I. Kolláth, B. Mosolygó, P. Hegedűs, and R. Ferenc, “Pybughive: A comprehensive database of manually validated, reproducible Python bugs,” *IEEE Access*, vol. 12, 2024.
- [10] C. S. Xia and L. Zhang, “Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2024.
- [11] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, “Is self-repair a silver bullet for code generation?” in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [12] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “Autocoderover: Autonomous program improvement,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2024.
- [13] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying LLM-based software engineering agents,” *arXiv preprint arXiv:2407.01489*, 2024.
- [14] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [15] Z. Z. Wang, A. Asai, X. V. Yu, F. F. Xu, Y. Xie, G. Neubig, and D. Fried, “Coderag-bench: Can retrieval augment code generation?” in *Findings of the Association for Computational Linguistics: NAACL*, 2025.
- [16] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. H. Chi, N. Schärli, and D. Zhou, “Large language models can be easily distracted by irrelevant context,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

- [17] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [18] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, “When not to trust language models: Investigating effectiveness of parametric and non-parametric memories,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [19] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of EMNLP-IJCNLP 2019*, 2019, pp. 3982–3992.